

Upstream review: two periodicity APIs

additive endomorphism geometric sums; finite-field Frobenius

Verdict

Both APIs are mathematically correct, small, and reusable. The Frobenius pair is essentially ready. For the endomorphism pair, replace the bespoke induction by specialization of `mul_geom_sum`; this removes `Mathlib.Tactic.Abel` and makes the intended API relationship explicit. I would upstream the pairs separately.

1. Additive endomorphisms

Use the ring theorem already imported — Change

The submitted proof re-proves the geometric identity by induction, although `Mathlib.Algebra.Ring.GeomSum` supplies

```
mul_geom_sum phi n :
  (phi - 1) * (sum i in range n, phi ^ i) = phi ^ n - 1
```

Evaluate this identity at `x`. The only coercion bridge needed is evaluation of the finite sum; package evaluation as an additive hom and use root-level `map_sum`:

```
let ev : AddMonoid.End A ->+ A :=
  { toFun := fun f => f x
    map_zero' := rfl
    map_add' := fun _ => rfl
    have hsum : ((sum i in range n, phi ^ i) : AddMonoid.End A) x =
      sum i in range n, (phi ^ i) x :=
      map_sum ev (fun i => phi ^ i) (range n)
    have h : := congrArg (fun f : AddMonoid.End A => f x)
      (mul_geom_sum phi n)
    change phi ((sum i in range n, phi ^ i) x -
      (sum i in range n, phi ^ i) x = (phi ^ n - 1) x at h
    rwa [hsum] at h
```

This is shorter conceptually, avoids duplicated algebra, and needs no tactic import.

Simplify the fixed-point corollary — Change

The current proof converts equality to subtraction twice through nested `simpa`. Prefer the visible one-line calculation:

```
rwa [← sub_eq_zero, apply_sum_pow_sub, hphi]
simp
```

The submitted proof is valid; this is a readability change.

Placement and names — Keep

Do not add the geometric-sum dependency directly to the foundational `Algebra.Group.Hom.End` file merely for two corollaries. A small file such as `Algebra/Group/EndomorphismsGeomSum.lean` is the clean dependency direction. The namespace and hypothesis `[AddCommGroup A]` are right. The existing names are discoverable from `AddMonoid.End`; an alternative aligned more tightly with the source lemma is `apply_mul_geom_sum`, but renaming is not required.

$$\begin{array}{c} (\phi - 1) \sum_{i < n} \phi^i = \phi^n - 1 \text{ in } \text{End}(A) \\ \uparrow \\ \text{evaluate at } x \\ \phi \left(\sum_{i < n} \phi^i x \right) - \sum_{i < n} \phi^i x = x - x \\ \uparrow \\ \phi^n = 1 \iff \phi \left(\sum_{i < n} \phi^i x \right) = \sum_{i < n} \phi^i x \end{array}$$

2. Finite-field Frobenius

The abstraction is exactly right — Keep

```
theorem frobenius_pow_eq_pow_mod
  (hcard : Fintype.card K = p ^ n) (m : Nat) :
  frobenius K p ^ m = frobenius K p ^ (m % n) :=
  pow_eq_pow_mod m (FiniteField.frobenius_pow hcard)
```

This is the natural generic corollary of `FiniteField.frobenius_pow`. Keep both the endomorphism equality and pointwise theorem: they serve different rewrite contexts. The proof of the pointwise theorem is already minimal and clear.

Drop the redundant import — Change

```
import Mathlib.FieldTheory.Finite.Basic
import Mathlib.GroupTheory.OrderOfElement
```

The second import is unnecessary: `pow_eq_pow_mod`, `FiniteField.frobenius_pow`, and `iterateFrobenius_eq_pow` are all available from `Mathlib.FieldTheory.Finite.Basic` in the audited Mathlib version. Keep only the first import. If declarations are inserted directly after `frobenius_pow` in that same file, no new import is needed at all.

No positivity hypothesis is needed — Keep

Do not add $0 < n$. The generic theorem handles $n = 0$, and the cardinality premise is already incompatible with a nontrivial finite field having cardinality $p^0 = 1$. The current statement is minimal and faithful.

Documentation and naming — minor

The module text says “extension degree.” The formal input is only a cardinality equality, so “exponent in the cardinality presentation” is maximally literal; “extension degree” is standard and acceptable in this finite-field context. `iterateFrobenius_mod` matches existing concise Frobenius naming. Add `eq` only if a maintainer requests strict equality-name style.

$$\begin{array}{c} |K| = p^n \xrightarrow{\text{frobenius_pow}} F^n = 1 \xrightarrow{\text{pow_eq_pow_mod}} F^m = F^{m \bmod n} \\ \downarrow \text{evaluate} \\ F^m(x) = F^{m \bmod n}(x) \end{array}$$

Suggested PR split

1. Endomorphism PR: new geometric-sum companion file; ring-level specialization; no `Tactic.Abel`.
2. Finite-field PR: place both lemmas immediately after `FiniteField.frobenius_pow`; no extra import.

Verification

All four declarations were compiled together against the pinned Mathlib version. The endomorphism proof was also compiled in its recommended ring-theorem form. No `sorry` is used.